bottleneck, as shown in figure 13.6: on the left half, you can see how, for basic parallelism, the “intermediate output” is all sent to the orchestrator, that has to gather it all and sort it to the machines in layer 2.

In *MapReduce*, instead, every step is intrinsically parallel: data is already broken down into pieces that can be processed independently, and results are routed to the reducers by each mapper on its own, without passing through a central orchestrator.

Well, to be exact, technically the reducers are the ones that go read the information from each mapper, while the mappers’ task is to create temporary files for each reducers in a specific location (different for each reducer: imagine, for instance, that each mapper creates a different folder or a dedicated virtual disk for each reducer).

Besides speed, MapReduce approach has another advantage: if a machine crashes, that single machine can be replaced without having to restart the whole computation. This, in turn, can also help us improve increase availability and reduce latency by allocating redundant resources to preventively cope with malfunctions.

One thing needs to be clear: in MapReduce as well there is an orchestrator, a master that controls the computation (spinning up the computational nodes, or requesting existing resources, assigning the input chunks to the masters, planning how to route intermediate results to reducers, and handling/recovering from errors). The difference with canonical parallelism, though, is that this special node doesn’t actually read the input nor compute it, and that